POLYSHIELD PRO

MACHINE-LEARNING-ASSISTED POLYMORPHIC MALWARE DETECTION: SYSTEM DESIGN, SYNTHETIC DATASET AND PROTOTYPE

UPDATED RESEARCH PROJECT REPORT

AUTHOR

AADHI KABILAN J

Updated using supplied source code, datasets, project notes,
application screenshot and the public GitHub repository README and source files.

November 2026

DOCUMENT CONTROL

Field	Description
Project	PolyShield Pro / AI Polymorphism Detection Simulation
Report author	Aadhi Kabilan J
Version	2.1 – author data-source clarification
Date	November 2026
Primary repository	https://github.com/AadhiKabilan/AI_Polymorphism_Detection_Sim
Evidence reviewed	Uploaded Python/C++ files, CSV datasets, README, ethics note, requirements file, model artefact filename and UI screenshot; public repository README and selected source files.

Authorship and provenance This report is authored by Aadhi Kabilan J. The earlier SRS/SDS lists multiple project contributors. This report names only the requested author, but does not imply that all original collaborative implementation work was performed solely by the report author.

ABSTRACT

PolyShield Pro is a machine-learning-assisted cybersecurity prototype developed to explore the detection of polymorphic-style changes through feature analysis and classification. The project combines (1) a desktop prototype with an 18-feature, eight-class Random Forest and LightGBM ensemble, and (2) a public GitHub simulation pipeline that generates synthetic JSON artefacts, perturbs feature records and trains a seven-feature binary Random Forest classifier. The author additionally confirms using a Microsoft malware dataset and feature datasets extracted from sample variants generated during the project. The supplied repository snapshot, however, does not establish the exact Microsoft dataset release or demonstrate that those external-sample features were ingested into the checked-in CSV files or model-training pipeline. These data sources are therefore described separately and transparently.

The supplied enhanced_polyshield_training_data.csv contains 500 rows across eight numeric classes, with class 7 represented by 100 rows. Inspection of the accompanying retraining script shows that these records are generated from random distributions rather than extracted from a verified corpus of executable samples. The separately supplied combined_features.csv contains 15,275 rows with two labels, benign and malware_sim; the associated simulator and variant synthesizer operate on synthetic JSON records and do not read or transform real binaries. The author has separately stated that Microsoft malware samples were also used; however, the exact Microsoft dataset identity, source, hashes, acquisition records and linkage to the supplied CSV files were not present in the reviewed evidence.

This report therefore distinguishes measured file facts, implementation behaviour, screenshot observations and author-reported context. The desktop screenshot displays 281 processes scanned, 56 threats detected, 8 polymorphic findings, 3 terminated processes and 81.1% model accuracy; these are treated as application-displayed counters, not independently reproduced benchmark results. The report also identifies important limitations in the training workflow, feature extraction, validation design and repository documentation. PolyShield Pro is positioned as an educational defensive prototype and not as a production malware detection product.

Keywords: polymorphic malware, malware analysis, synthetic dataset, Random Forest, LightGBM, feature engineering, static analysis, cybersecurity, Flask, Tkinter.

EXECUTIVE SUMMARY

Implementation	What the evidence shows	How it should be described
Desktop PolyShield Pro FINAL	Tkinter GUI; 18 engineered features; numeric classes 1–8; Random Forest + LightGBM soft voting; heuristic; psutil process termination.	A local prototype with a distinct training/inference pipeline.
GitHub simulation repository	Synthetic JSON artefact generator; JSON variant perturbation; 7-feature CSV; binary benign/malware_sim label; Random Forest + heuristic; Flask dashboard.	An educational simulation of polymorphic-style feature variation; it does not process real binaries.
500-row enhanced training CSV	500 rows, 18 numeric input features plus class; 8 class labels; generated by random ranges in the supplied retraining script.	Synthetic training data; not a 500-sample real malware corpus.
15,275-row combined CSV	15,275 records; 9 data/identifier columns plus binary label; synthetic JSON pipeline output.	Synthetic records and augmented variants; not executable samples.
Microsoft malware samples	Reported by the project author, but source details and sample-level provenance were not supplied.	Mention only with a precise dataset citation and clear separation from synthetic records.

Main publication recommendation Publish the GitHub simulation as a synthetic-data research prototype. Keep the desktop GUI as a separate project/implementation track unless you integrate the code, model, feature schema and documentation into one reproducible repository.

1. INTRODUCTION

1.1 Background

Polymorphic malware changes its code representation while attempting to preserve its underlying behaviour. Changes to byte sequences, encoding, structure or surrounding code can weaken simple signature matching. Detection systems therefore often combine multiple signals, including static file characteristics, API references, entropy, structural metadata and behavioural observations.

1.2 Motivation

The project explores how machine-learning models and explicit heuristics can be used to classify suspicious patterns associated with polymorphic-style variation. It also provides an interactive demonstration of how changes in section entropy, file structure, imported API counts, strings and junk-byte indicators can affect a classifier's output.

1.3 Project objectives

- Generate synthetic artefacts and derive perturbed variants for repeatable experimentation.
- Represent artefacts through structured numerical features.
- Train supervised classifiers and compare their predictions with a rule-based heuristic.
- Expose predictions, metrics and dataset summaries through a dashboard or desktop interface.
- Document the limits of synthetic evaluation and establish a safe boundary for any future real-sample research.

1.4 Scope

The GitHub repository scope is JSON/CSV feature simulation and classification. It does not create, modify, compile or execute real binaries. The separate desktop application reads up to the first 1 MB of a selected file as bytes and counts textual patterns in the decoded content; the supplied code does not establish a full PE parser/disassembler pipeline. The distinction matters when describing the project as static malware analysis.

2. RELATED TECHNICAL CONCEPTS

2.1 Signature detection and polymorphism

Signature detection compares artefacts against known byte patterns or indicators. Polymorphic transformations can change the observable representation and reduce the usefulness of exact matching. Feature-based approaches attempt to identify broader patterns, but can also be evaded and can produce false positives.

2.2 Static feature analysis

Static feature analysis extracts properties without executing the sample. Examples include file size, section metadata, entropy, imported API counts and string counts. Such features can support triage, but do not prove maliciousness independently.

2.3 Entropy

Shannon entropy measures uncertainty in a byte distribution. For byte values, the theoretical range is 0–8 bits per byte. High entropy may be consistent with encryption or packing, but also occurs in legitimate compressed content.

2.4 Supervised learning and rule fusion

Supervised learning maps feature vectors to known labels. Rule fusion adds deterministic conditions to model output. Both approaches require independent validation and representative labels; a high score on synthetic data does not establish generalisation to real-world malware.

3. IMPLEMENTATION TRACK A – DESKTOP POLYSHIELD PRO FINAL

3.1 Application overview

The supplied polyshield_final_termination.py file implements a Windows-oriented Tkinter desktop application. It includes a dark dashboard interface, model loading, quick scanning, protection monitoring, event display, statistics and optional process termination through psutil. The screenshot provided by the author shows the application branded “PolyShield Pro FINAL” with dashboard cards and a process/threat table.

3.2 Desktop feature schema

Feature	Implementation-level meaning from supplied extractor
file_size	File length in bytes.
entropy	Shannon entropy of the bytes read.
api_count	Occurrences of configured suspicious API name strings.
string_count	Count of quote characters in decoded file content; not a parser-derived count of strings.
function_count	Occurrences of simple tokens such as function, proc, def, void, int and main(.
xor_operations	Occurrences of XOR-related textual patterns.
memory_alloc_calls	Occurrences of allocation-related substrings.
thread_creation	Occurrences of thread-related substrings.
encryption_functions	Occurrences of configured encryption-related keywords.
shellcode_indicators	Count of escaped hexadecimal byte patterns and selected weighted substrings.
suspicious_apis	Count of configured suspicious API-name strings.
obfuscation_level	Count of configured obfuscation-related words and identifier fragments.
control_flow_complexity	Count of keywords such as if, for, while, switch, goto, call and volatile.
messagebox_calls	Occurrences of the word messagebox.
has_encryption	Binary flag based on encryption keyword count.
has_obfuscation	Binary flag based on obfuscation keyword count.
api_diversity	Capped API count divided by 10; this is not a true unique-API diversity ratio.
complexity_score	Average of function, control-flow and obfuscation counts.

The extractor uses substring counting over a decoded byte string. It is therefore a heuristic text scanner, not a PE import-table parser or instruction-level disassembler. The feature names should be described as indicators, not as authoritative measurements of executable semantics.

3.3 Classification pipeline

The retraining script defines a StandardScaler, LabelEncoder, SelectKBest using the ANOVA F-test, Random Forest and LightGBM models. The ensemble is a VotingClassifier configured for soft voting. Random Forest is configured with 200 trees and max_depth=12; LightGBM is configured with 200 estimators, learning_rate=0.1, num_leaves=31 and max_depth=8. The script saves a model bundle containing the ensemble, scaler, feature selector, label encoder and feature names.

3.4 Eight-class training data

The supplied enhanced_polyshield_training_data.csv contains exactly 500 rows and 19 columns: 18 numeric predictors and the class target. Its class distribution is:

Class	Rows	Share
1	80	16%
2	70	14%
3	70	14%
4	60	12%
5	50	10%
6	40	8%
7 – polymorphic	100	20%
8	30	6%
Total	500	100%

The supplied retraining script creates these class patterns using numpy.random.uniform ranges. It generates 50 records for each of classes 1–6, 100 records for class 7 and 50 records for class 8. The CSV class counts show 30 records for class 8, so the checked-in CSV does not exactly match the visible loop counts in the supplied script; this should be reconciled before describing the dataset-generation run. The class distributions and feature ranges are synthetic and should not be labelled as observed measurements from real malware.

3.5 Training and validation limitation

Critical methodological issue In retrain_polyshield_model.py, StandardScaler and SelectKBest are fitted to the full dataset before train_test_split. This leaks information from the test partition into preprocessing. The split must occur first; then fit the scaler and selector only on X_train, and transform X_test with those fitted objects.

The script reports a test accuracy and classification report, but the dataset is synthetic and the preprocessing order is not methodologically sound for an unbiased test estimate. Any accuracy from this run must be labelled an internal synthetic-data result, not a real-world malware detection rate.

3.6 Heuristic

The desktop retraining notes describe a post-prediction rule that assigns class 7 when encryption and obfuscation indicators are both present and shellcode indicators exceed a configured threshold. The guide gives 10 as an example threshold. The exact operational threshold and confidence calculation should be stated from the final source code and tested independently. A rule-forced score should not be presented as a calibrated probability.

4. IMPLEMENTATION TRACK B – GITHUB SYNTHETIC SIMULATION

4.1 Repository identity and stated purpose

The public repository is named AI_Polymorphism_Detection_Sim. Its README describes an educational research prototype that uses synthetic JSON artefacts only. It states that no real binaries are created, modified, compiled or executed, and that all artefacts are JSON records containing PE-like feature fields.

Repository: https://github.com/AadhiKabilan/AI_Polymorphism_Detection_Sim

4.2 Synthetic base artefact generation

The simulator creates a random number of sections, section sizes, section entropy values, imported API counts, string counts, entry-point and image-base values, junk-byte counts and resource sizes. It assigns one of two labels: benign or malware_sim, with configured sampling weights of 60% and 40%. Each record is saved as JSON with a generated UUID.

4.3 Variant synthesis

The public variant synthesizer reads JSON records and creates perturbed copies. It changes the record identifier, preserves an origin_id link, perturbs section entropy with bounded Gaussian noise, shuffles or splits section sizes, perturbs junk-byte counts, imported API counts, strings counts and resource size, and preserves the source label by default. This is feature-record augmentation, not binary-level polymorphism: it does not rewrite executable code.

4.4 Dataset construction

The dataset builder reads JSON files from the base-artifact and variant directories and flattens each record into a CSV row. The visible implementation outputs identifiers, section count, average section entropy, total section size, imported API count, string count, junk-byte count, resource size and label. The origin_id field supports traceability between a base record and its perturbed descendants.

4.5 Supplied combined dataset

The uploaded combined_features.csv contains 15,275 rows and 10 columns. It has 10,726 benign rows and 4,549 malware_sim rows. The origin_id column contains 105 missing values. This is consistent with a mixture of base records and derived records, but the precise generation parameters and complete lineage of this CSV should be preserved in a dataset manifest.

Property	Observed value
Rows	15,275
Columns	10
Labels	benign; malware_sim
benign	10,726 (approximately 70.2%)
malware_sim	4,549 (approximately 29.8%)
Missing origin_id	105 rows
Data representation	JSON-derived numeric feature records; not binary files

4.6 GitHub model

The public detection/train_model.py uses seven input features: num_sections, avg_section_entropy, total_section_size, imported_api_count, strings_count, junk_bytes_count and resource_size. It maps benign to 0 and malware_sim to 1, then trains a RandomForestClassifier with 200 trees. An additional rule flags records when

junk_bytes_count exceeds 2,500 and average section entropy exceeds 6.0. The final positive decision is the logical OR of the ML prediction and the heuristic.

The training script performs a stratified 80/20 train/test split before model fitting. It reports accuracy, precision, recall and a confusion matrix and saves a model bundle and metrics JSON. The README also documents a Flask dashboard with Home, Metrics, Classify, Playground and Dataset pages.

4.7 The two model pipelines are not interchangeable

Property	Desktop PolyShield Pro FINAL	GitHub simulation
Input	File bytes / process-related scan target	Synthetic JSON records / CSV features
Feature count	18	7
Target labels	Eight numeric classes 1–8	Two labels: benign / malware_sim
Models	Random Forest + LightGBM soft voting	Random Forest
Heuristic	Encryption + obfuscation + shellcode indicator condition	Junk-byte + average-entropy condition
Interface	Tkinter desktop GUI	Flask web dashboard
Process termination	Included through psutil	Not part of documented GitHub pipeline

5. AUTHOR-REPORTED MICROSOFT MALWARE SAMPLE USE

The author confirms that the project work used a Microsoft malware dataset and also used feature datasets extracted from sample variants generated by the author. These are two distinct sources of experimental data: (a) malware samples associated with a Microsoft dataset, and (b) feature records derived from the author-generated sample variants. The supplied repository snapshot and CSV files reviewed for this report do not contain enough provenance metadata to identify the exact Microsoft dataset release, quantify the samples, or trace those external-sample features into a particular checked-in CSV or model run. Accordingly, this report records the author-confirmed use without inventing a dataset title, sample count, licence, or performance result.

5.1 Required provenance record

5.1.1 Author-confirmed data sources

Microsoft malware dataset: used in the project according to the author. The exact dataset title, release/version, sample count, label mapping and licence were not specified in the materials available for this revision.

Author-generated variant features: the author generated sample variants and extracted feature records from selected variants for use as an additional dataset. The supplied materials do not specify the exact number of variants, feature-extraction configuration or the precise CSV/model run into which these records were incorporated.

- For the Microsoft source, add the exact dataset title, publisher/host page and official citation. The current project materials identify it only as a Microsoft malware dataset.
- Record the version/release date, access or download URL, and licence/terms of use for the Microsoft dataset.
- Record the number of Microsoft samples acquired, retained and excluded, with exclusion reasons; separately record the number of author-generated variants and extracted feature records.
- Record sample hashes and source labels in a private or permitted manifest; do not publish malware binaries or restricted identifiers.
- Document whether Microsoft samples were analysed statically, dynamically or only used to derive metadata/features. For author-generated variants, describe the variant-generation procedure and feature-extraction method.
- Provide a traceable mapping from source sample/variant IDs to extracted feature records and train/test partitions, while keeping related variants grouped during evaluation.
- Confirm that no malware binaries are committed to the public repository unless redistribution is explicitly permitted.

5.2 Safe publication language

Publication wording: “The project combines synthetic feature simulation with an author-reported Microsoft malware dataset and feature records extracted from author-generated sample variants. The public GitHub simulation pipeline currently documents synthetic JSON feature artefacts; the checked-in materials do not yet provide a complete provenance manifest or establish which external-sample feature records were included in each reported model-training run.” Do not claim that the GitHub synthetic pipeline directly trains on Microsoft samples unless the code and dataset lineage demonstrate that connection.

6. USER INTERFACE AND OBSERVED DEMONSTRATION

The supplied screenshot shows the desktop application header, Load Model, Quick Scan and Start Protection controls, summary cards and a threat table. The visible cards show 281 processes scanned, 56 threats detected, 8 polymorphic findings, 3 processes terminated, model accuracy of 81.1%, and a last-update date of 2025-09-19.

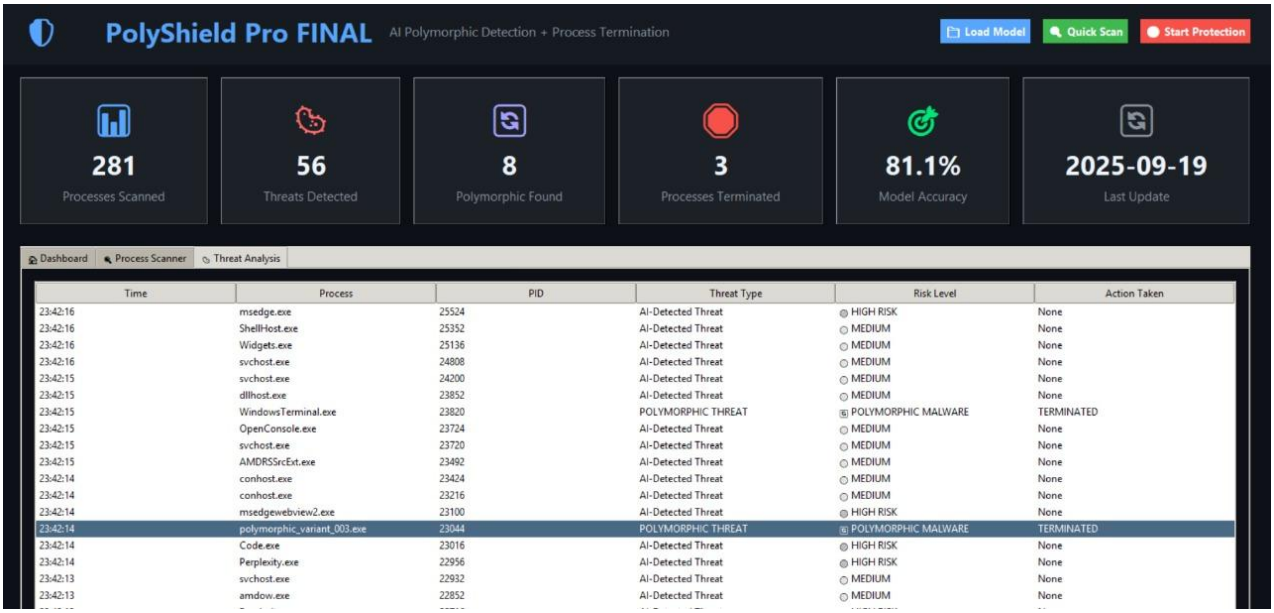


Figure 1. PolyShield Pro FINAL desktop dashboard screenshot supplied by the author.

These values are screenshot observations only. The screenshot does not establish the test dataset, ground-truth labels, measurement procedure or reproducibility of the displayed model accuracy. The 81.1% figure must not be represented as a verified benchmark until supported by saved test metrics and a documented evaluation protocol.

7. TESTING AND EVALUATION

7.1 Metrics to report

Metric	Purpose
Accuracy	Overall fraction of correct predictions.
Precision	Reliability of positive predictions.
Recall	Fraction of actual positive samples detected.
F1-score	Harmonic mean of precision and recall.
Confusion matrix	Shows true and predicted class counts.
False-positive rate	Benign samples incorrectly flagged as threats.
Latency	Separate feature extraction time from model inference time.
Family/source holdout	Tests whether the model generalises beyond related synthetic variants.

7.2 Evaluation design recommended

- Split data before fitting any scaler, selector or model.
- Keep all variants derived from the same origin_id in one partition to avoid parent/child leakage.
- Use a fixed random seed and record all package versions and configuration.
- Report per-class precision, recall and F1 for the eight-class desktop pipeline.
- For the binary simulation, report the positive class definition and confusion matrix.
- Evaluate the ML-only model, heuristic-only rule and combined decision separately.
- Use a genuinely independent dataset for any claim about real-world malware detection.
- Benchmark inference on documented hardware and repeat measurements.

7.3 Results status

Result or claim	Evidence status
500-row, eight-class CSV	Verified from supplied CSV; synthetic feature data.
Class 7 has 100 rows	Verified from supplied CSV.
15,275-row binary CSV	Verified from supplied CSV; JSON-derived synthetic records.
81.1% desktop model accuracy	Visible in screenshot; not independently validated.
95% class-7 detection target	Appears in earlier SRS as a target, not a measured result.
Microsoft malware sample use	Author-confirmed use of a Microsoft malware dataset; exact dataset identity, sample count and linkage to checked-in model runs are not documented in the supplied snapshot.

8. REPOSITORY REVIEW AND QUALITY AUDIT

The public README, simulator, variant synthesizer, dataset builder, training script and Flask dashboard were reviewed through the publicly accessible raw GitHub files. The separate uploaded desktop code and CSVs were inspected locally. This is a source-level review of the files available in this conversation, not a full GitHub Actions run, dependency vulnerability scan or complete repository clone.

8.1 What is good

- The repository README clearly states the synthetic-only scope and safety boundary.
- The simulator and variant synthesizer operate on JSON feature records rather than executable binaries.
- The variant synthesizer preserves origin_id, which can support lineage-aware train/test splitting.
- The CSV builder provides a repeatable conversion from JSON records to a tabular dataset.
- The binary training script uses stratified train/test splitting and reports precision, recall, accuracy and a confusion matrix.
- The Flask dashboard separates metrics, classification, playground and dataset views.
- The ethics note explicitly discourages testing malware on networked or vulnerable systems.

8.2 Issues to fix before public release

Priority	Finding	Recommended action
High	The desktop app and GitHub repository describe different schemas and architectures.	Clearly separate them in README and report; do not imply the GitHub app runs the desktop model.
High	The desktop retrainer fits StandardScaler and SelectKBest before the train/test split.	Split first; fit preprocessing on training data only; evaluate untouched test data.
High	The 500-row CSV is synthetically generated from class-specific random ranges.	Label it synthetic; do not claim it is a Microsoft malware dataset or real malware corpus.
High	The GitHub variant synthesizer changes feature records, not executable code.	Call it synthetic feature-level polymorphic simulation, not binary polymorphism.
High	The public README describes files and folders not included in the uploaded file set.	Verify that every documented path exists in the actual repository and that the quickstart works from a clean clone.
Medium	The dataset builder assumes all required output columns exist and may fail on empty or incomplete input.	Validate input schema, handle empty datasets and report missing/invalid fields clearly.
Medium	The simulator uses unseeded global randomness.	Add a CLI seed option and record the seed for reproducibility.
Medium	No test suite or automated CI evidence was included in the reviewed upload.	Add unit tests for generation, lineage, dataset building, model inference and dashboard routes.
Medium	requirements.txt uses unpinned packages.	Pin compatible versions or add a lockfile and test a clean installation.
Medium	The Flask dashboard loads a serialized joblib model.	Treat model files as trusted-only; never load user-uploaded model artefacts.
Medium	The dashboard uses render_template_string with a body value marked safe in the page wrapper.	Avoid marking dynamic content safe; use template files and escaped variables.
Medium	No explicit upload size limit or CSRF protection was evident in the reviewed dashboard routes.	Set request-size limits and add appropriate web security protections before deployment.

8.3 Desktop implementation findings

- The extractor reads raw file bytes and counts substrings; it does not demonstrate a full PE import parser despite an optional pefile import.
- The api_diversity calculation is min(api_count / 10, 1), not unique API count divided by total API count.
- The function, control-flow, memory and obfuscation features are naive substring counts and may count incidental text or comments.
- On feature extraction exceptions, the desktop extractor returns all-zero features. This can silently produce misleading predictions; errors should instead be surfaced and the scan marked failed.
- The process terminator calls terminate(), waits, and may then kill(). This can interrupt legitimate processes if a false positive occurs; keep termination disabled by default and require confirmation.
- The model accuracy display path uses a fallback value when accuracy metadata is unavailable; the displayed number must be sourced from actual evaluation metadata, never a default.
- The supplied C++ demonstration variants contain API misuse and inconsistent transform/de-transform operations. For example, LoadLibrary is used as though it were a memory-allocation function, and some decryption routines do not reverse their corresponding encryption operation. Treat them as unvalidated examples, not tested executable samples.

8.4 GitHub publication checklist

- Ensure README accurately distinguishes synthetic JSON simulation from the separate desktop app.
- Add a clear project status and limitations section near the top.
- Add a LICENSE only if you have the right to license all included code and assets.
- Add .gitignore entries for .venv, __pycache__, *.pyc, generated sample artefacts, local model files, logs and private datasets.
- Do not commit real malware binaries, proprietary Microsoft samples, personal data or restricted datasets.
- Include a dataset card documenting synthetic generation, labels, feature definitions, class balance and limitations.
- Include a reproducible experiment command and save the random seed and metrics.
- Add tests and a GitHub Actions workflow for linting and tests.
- Pin dependencies and review package compatibility.
- Add SECURITY.md explaining responsible reporting and safe use.
- Add a screenshot or GIF of the Flask dashboard, with a caption stating that it displays synthetic-data results.
- Remove stale references to files that are not present or add the missing files and verify all documented commands.

9. SECURITY, ETHICS AND DATA GOVERNANCE

The project is intended for educational and defensive research. The synthetic GitHub pipeline should remain the default reproducible demonstration. Any future real-sample analysis should be authorised, conducted in an isolated lab, and performed using static analysis or a controlled sandbox under institutional procedures. Samples must not be executed on a normal workstation or open network.

For real sample use, preserve provenance, hashes, acquisition records, labels, licence conditions and chain-of-custody details. Do not redistribute Microsoft or third-party malware samples unless the dataset terms explicitly permit redistribution. The public repository should contain code, synthetic fixtures and metrics only, unless real sample publication is expressly authorised.

Automatic process termination is a potentially disruptive response. A safe default is alert-only mode. Any termination action should require explicit opt-in, show the process name and PID, record the reason and outcome, and avoid critical system processes.

10. LIMITATIONS

- Synthetic feature distributions are not a substitute for a representative real-world malware corpus.
- Feature-level perturbation does not reproduce binary-level polymorphism or runtime behaviour.
- The 500-row eight-class dataset and 15,275-row binary dataset belong to different data schemas and experiments.
- The desktop training script has preprocessing leakage before splitting.
- The desktop feature extractor uses heuristic substring counts and may not represent actual PE structure.
- The screenshot metrics are not backed by an independent evaluation report in the supplied evidence.
- The Microsoft dataset attribution is not reproducible without exact source and sample-level provenance.
- The repository review did not execute the project or verify all files in a clean environment.

11. FUTURE WORK

- Unify project naming and document the two implementation tracks separately.
- Create a single documented feature schema for each pipeline and prevent accidental model interchange.
- Correct preprocessing leakage and add source/family-aware data splits.
- Add deterministic generation through seeded random-number generators.
- Add automated tests for the simulator, dataset builder, classifier, dashboard and desktop extractor.
- Add robust PE metadata extraction using a validated parser, while keeping the synthetic pipeline separate.
- Add a documented, authorised external evaluation dataset and report provenance and licence.
- Evaluate the model-only, heuristic-only and combined approaches independently.
- Calibrate probabilities and report class-wise metrics and false-positive rates.
- Use alert-first process monitoring with explicit user approval for termination.

12. CONCLUSION

PolyShield Pro demonstrates two complementary educational approaches to polymorphic-malware-related detection. The desktop application explores an 18-feature, eight-class classifier with a Random Forest/LightGBM ensemble, a rule-based polymorphic condition and process-monitoring functions. The public GitHub repository provides a safer and more reproducible synthetic-data simulation in which JSON artefacts are generated, perturbed, flattened into a CSV, classified by a Random Forest model and explored through a Flask dashboard.

The reviewed evidence supports the claim that the project contains a 500-row synthetic eight-class training dataset and a separate 15,275-row synthetic binary dataset. It does not support a claim that these datasets were extracted from Microsoft malware samples. The author-reported use of Microsoft samples should be documented separately with exact provenance before it is presented as part of the experimental dataset. The desktop training pipeline also requires a correction to avoid preprocessing leakage, and the repository README should be checked against the actual committed file tree.

With these corrections, the project can be presented credibly as an educational research prototype that demonstrates synthetic feature simulation, machine-learning classification and heuristic-assisted detection, while transparently acknowledging that production-level malware detection performance has not yet been established.

REFERENCES AND PROJECT MATERIALS

[1] Aadhi Kabilan J, AI_Polymorphism_Detection_Sim, public GitHub repository, https://github.com/AadhiKabilan/AI_Polymorphism_Detection_Sim (accessed November 2026).

[2] Repository README.md, describing synthetic JSON artefacts, quickstart, dashboard and ethics scope.

[3] Repository artifact_simulator/simulator.py, synthetic base-artifact generation implementation.

[4] Repository variant_synthesizer/synthesize_variants.py, JSON feature perturbation implementation.

[5] Repository dataset/build_dataset.py, JSON-to-CSV feature aggregation implementation.

[6] Repository detection/train_model.py, Random Forest and heuristic training/evaluation implementation.

[7] Repository dashboard/app.py, Flask dashboard implementation.

[8] PolyShield Pro, Software Requirements Specification (SRS) and Software Design Specification (SDS), supplied project document, 20 September 2025.

[9] PolyShield Pro, Comprehensive Guide to Polymorphic Malware Simulation, Detection Features, and Machine Learning Algorithms, supplied project guide.

[10] Supplied source files: retrain_polyshield_model.py, polyshield_final_termination.py, simulator.py, build_dataset.py and sample C++ demonstration files.

[11] Supplied datasets: enhanced_polyshield_training_data.csv and combined_features.csv.

[12] Supplied ethics note and PolyShield Pro FINAL dashboard screenshot.

APPENDIX A – DATASET SUMMARY

Dataset	Rows	Inputs / columns	Labels	Evidence classification
enhanced_polyshield_training_data.csv	500	18 numeric predictors + class	1–8	Synthetic class-wise random feature data
combined_features.csv	15,275	8 feature fields + id + origin_id	benign / malware_sim	Synthetic JSON records and perturbed variants
Author-generated variant feature records	Not specified	Extracted feature vectors from sample variants generated by the author	Not specified	Author-confirmed source; exact count, schema and CSV/model linkage not supplied
Microsoft malware dataset	Not specified	Features/metadata used in project; exact schema not supplied	Not specified	Author-confirmed use; exact dataset identity and experiment linkage not documented

APPENDIX B – GITHUB QUICKSTART AS DOCUMENTED

The repository README documents the following workflow. Run it only after verifying that the named files exist in the repository and that the environment is isolated.

1. Create a Python virtual environment and install requirements.txt.
2. Generate synthetic base JSON artefacts using artifact_simulator/simulator.py.
3. Generate perturbed JSON variants using variant_synthesizer/synthesize_variants.py.
4. Build dataset/combined_features.csv using dataset/build_dataset.py.
5. Train the Random Forest model using detection/train_model.py.
6. Launch dashboard/app.py locally and inspect Metrics, Classify, Playground and Dataset pages.

APPENDIX C – DATASET PROVENANCE DETAILS TO COMPLETE FOR FULL REPRODUCIBILITY

7. Exact Microsoft dataset name and official URL.
8. Whether the samples are from a Microsoft research dataset, a Microsoft-hosted benchmark, or a third-party dataset mirrored by Microsoft.
9. Number of samples actually downloaded and analysed.
10. Sample hashes and label mapping.
11. Whether samples contributed to the 500-row CSV, the 15,275-row CSV, another dataset, or only manual testing.
12. Which feature extractor processed them and whether the process was static-only.
13. Permission/licence conditions for public redistribution or citation.

TECHNICAL AUDIT ADDENDUM – ADDITIONAL REPRODUCIBILITY FINDINGS

This addendum records two additional findings from reviewing the public GitHub data-generation and training source files. They should be addressed before using the repository's metrics as evidence of classifier performance.

A.1 Synthetic label-generation problem

The base simulator generates feature values first and assigns the benign/malware_sim label independently using random sampling weights (60%/40%). The visible code does not define class-conditional feature distributions or a deterministic relationship between the generated feature values and the assigned label. The variant synthesizer then preserves the original label while perturbing feature values. Consequently, the current synthetic dataset does not establish a meaningful ground-truth relationship between the feature vector and malware_sim label. A classifier may learn noise or incidental correlations rather than a defensible malware signal.

A.2 Parent/variant leakage in evaluation

The variant synthesizer records origin_id to identify the parent record, but the GitHub training script uses a row-level stratified train_test_split and does not group by origin_id. A base artefact may therefore be placed in the training set while one or more perturbed descendants are placed in the test set. This can make evaluation optimistic because related records appear on both sides of the split. Use a group-aware split keyed by origin_id (with base records assigned their own identity) and report performance on held-out parent groups.

A.3 Required correction sequence

14. Define a defensible synthetic ground-truth rule or label-generation process that creates a known relationship between feature patterns and the target class.
15. Document that the synthetic labels represent simulated categories, not verified real malware verdicts.
16. Split by parent/source group before model fitting; keep all variants from one origin in the same partition.
17. Fit any preprocessing or feature selection only on the training partition.
18. Compare against simple baselines and report per-class metrics and the confusion matrix.
19. Keep a separate, provenance-documented real-world evaluation set if making claims about real malware.

Publication impact: Until these issues are corrected, metrics from the GitHub synthetic dataset should be presented as demonstration outputs only. They should not be used to claim that the model has learned to detect real polymorphic malware.